SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – SCENARIO BASED ASSESSMENT

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	**Assessment:**	Assessment Tool 2 – Scenario Based Assessment
Weightage:	20% of CIA	**Total Marks:**	100
CO2 Statement:	*Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)*		
Student Name:	J KARTHIK	**Reg. No.:**	192472136
Section / Batch:	CSE – AI / 2024	**Date:**	

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given scenario and identify the computational problem involved.
- Apply appropriate Brute Force techniques for sorting, searching, Closest-Pair, and Convex-Hull problems wherever applicable.
- Clearly justify the chosen solution approach with proper reasoning and analytical interpretation.
- Demonstrate decision-making skills by comparing possible solutions and selecting the most suitable approach.
- Use diagrams, stepwise illustrations, or algorithmic representations wherever necessary.
- Maintain clarity, logical organization, and proper technical presentation throughout the answers.

Question Type	Focus Area	Questions
Scenario Based Assessment	Analyze real-world scenarios and apply Brute Force techniques for sorting, searching, Closest-Pair, and Convex-Hull problems	Q1

SECTION A – SCENARIO BASED ASSESSMENT

Code of Conduct

*I **J KARTHIK / 192472136** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: J KARTHIK

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25%	Thoroughly understands the scenario with accurate interpretation of all requirements	Clearly understands the scenario with minor gaps	Basic understanding; some aspects of the scenario unclear	Poor understanding or misinterpretation of the scenario
Application of Concepts	30%	Applies appropriate concepts effectively to solve complex real-world problems	Applies relevant concepts correctly with minor errors	Applies basic concepts but with limited accuracy	Fails to apply appropriate concepts
Analytical & Decision-Making Skills	20%	Demonstrates strong analysis and selects optimal solutions with justification	Good analysis with reasonable solutions	Limited analysis; solutions lack justification	Poor or no analysis; incorrect decisions
Solution Design & Approach	15%	Well-structured, innovative, and feasible solution approach	Logical and structured solution with minor gaps	Basic solution with limited structure or feasibility	Unclear or incorrect solution approach
Clarity, Justification & Presentation	10%	Clear, well-justified explanation with strong reasoning	Clear explanation with some justification	Limited clarity and weak justification	Poorly explained with no justification

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25%				
Application of Concepts	30%				
Analytical & Decision-Making Skills	20%				
Solution Design & Approach	15%				
Clarity, Justification & Presentation	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Question 75: Malware Detection System (Brute-Force String Matching)

Scenario: A malware detection tool scans system logs using brute-force string matching to find suspicious patterns.

a) How Brute-Force String Matching Works

The brute-force (naive) string matching algorithm compares a given pattern P (the suspicious signature, length m) against the text T (the system log, length n) by aligning the pattern at every possible starting position in the text.

Working steps:

- Place the pattern P at position i = 0 of the text T.
- Compare P with the corresponding m characters of T, character by character.
- If all m characters match, a suspicious pattern occurrence is reported at position i.
- If a mismatch occurs, shift the pattern one position to the right (i = i + 1) and repeat the comparison from the start of P.
- Continue until the pattern has been checked at every position from i = 0 to i = n − m.

b) Time Complexity Analysis

Best Case	O(n) — mismatch occurs on the first character at most positions
Worst Case	O(n·m) — every position requires up to m comparisons (e.g., highly repetitive text/pattern)
Average Case	O(n) for typical natural-language / log text with few false starts
Space Complexity	O(1) — no extra data structures beyond the pattern and text

c) Why Performance Degrades on Large Logs

System logs are typically very large (n is huge) and are scanned repeatedly for many malware signatures. Brute-force matching re-examines overlapping substrings without remembering any information from previous comparisons, so:

- Every position in the log (up to n − m + 1 positions) requires a fresh comparison pass, even when partial matches share common prefixes.
- Repetitive or low-entropy log content (e.g., long runs of similar characters) triggers many partial matches before a mismatch, pushing performance toward the O(n·m) worst case.
- Scanning for multiple malware signatures multiplies the cost further, since each signature requires an independent O(n·m) scan.

This inefficiency justifies moving to optimized pattern-matching algorithms such as KMP, Boyer–Moore, or Rabin–Karp, which avoid redundant comparisons and achieve O(n + m) performance — essential for real-time malware/log scanning at scale.

Question 82: Telecom Signal Tower Optimization (Brute-Force Closest Pair)

Scenario: A telecom system evaluates distances between towers to optimize signal strength using brute-force closest pair computation.

a) How the Brute-Force Method Works

Given n tower coordinates plotted on a 2D plane, the brute-force closest-pair method systematically computes the Euclidean distance between every possible pair of towers and retains the pair with the minimum distance.

Working steps:

- Take all towers T1, T2, ..., Tn with coordinates (x_i, y_i).
- For every pair (T_i, T_j) where $i < j$, compute distance = $\sqrt{[(x_j - x_i)^2 + (y_j - y_i)^2]}$.
- Keep track of the minimum distance found so far and the corresponding pair of towers.
- After examining all C(n,2) pairs, the recorded pair with the smallest distance is the closest pair — used to detect towers causing signal overlap/interference.

b) Derivation of Time Complexity

Number of unique pairs to check = $C(n,2) = n(n-1)/2$

Each pairwise distance computation takes constant time O(1).

Total time = $O(1) \times n(n-1)/2 = O(n^2)$

Time Complexity	$O(n^2)$
Space Complexity	O(1) extra (excluding input storage)

c) Performance Limitations

As the telecom network expands and more towers are added, the quadratic $O(n^2)$ growth becomes a serious bottleneck:

- Doubling the number of towers quadruples the number of distance computations required.
- Real-time signal optimization needs frequent recomputation as towers are added/relocated, making repeated $O(n^2)$ scans computationally expensive.
- For large-scale telecom networks (thousands of towers), this approach does not scale, motivating the use of the divide-and-conquer closest-pair algorithm, which reduces the complexity to O(n log n).

Question 89: Payroll Employee ID Lookup (Binary Search)

Scenario: A payroll system uses Binary Search on sorted employee IDs, but frequent insertions disturb the order.

a) How Binary Search Works

Binary Search operates on a sorted array by repeatedly dividing the search interval in half:

- Set low = 0 and high = n − 1.
- Compute mid = (low + high) / 2 and compare the employee ID at index mid with the target ID.
- If they match, the search succeeds and returns index mid.
- If the target is smaller, discard the right half and set high = mid − 1; if larger, discard the left half and set low = mid + 1.
- Repeat until the target is found or the search interval becomes empty (low > high).

b) Impact of Unsorted Data on Performance

When order is maintained	O(log n) per search — each comparison eliminates half the remaining records
When order is disturbed	Binary Search produces incorrect or missed results, since its correctness depends entirely on sorted order
Required fix	Data must be re-sorted before searching, costing O(n log n) per re-sort

	using an efficient sorting algorithm

If insertions happen frequently and the array is naively re-sorted after every insertion, the effective cost per insertion becomes O(n log n) instead of the O(log n) that Binary Search promises for lookup alone — negating its efficiency advantage.

c) Importance of Maintaining Sorted Order

Binary Search's O(log n) efficiency is only achievable when the underlying data remains sorted at all times. For a payroll system with frequent insertions, this means:

- New employee ID insertions should use an insertion strategy that preserves order (e.g., insertion into a sorted array/list at the correct position, or using a self-balancing structure such as a B-Tree/AVL tree) rather than appending and re-sorting.
- Alternatively, if insertions are very frequent, a hashing-based lookup structure (O(1) average) may be more suitable than maintaining strict sorted order.
- Choosing the right data structure is a trade-off between insertion cost and lookup cost — critical for payroll systems that must guarantee fast, correct ID lookups.

Question 94: Disaster Response Zone Mapping (Brute-Force Convex Hull)

Scenario: A disaster response application maps affected areas using coordinate points and determines the boundary using convex hull techniques.

a) How Brute-Force Convex Hull Operates

The brute-force convex hull algorithm identifies the smallest convex polygon enclosing a set of n coordinate points (affected-area markers) by testing every pair of points as a potential hull edge.

Working steps:

- For every pair of points (P_i, P_j), form the line passing through them.
- Check all remaining n − 2 points to see whether they all lie on the same side of this line.
- If all other points lie on one side (or on the line), then the segment P_iP_j is an edge of the convex hull.
- Repeat for every pair of points; the collected set of valid edges forms the convex hull boundary — representing the outer perimeter of the disaster-affected region.

b) Computational Complexity

Pairs of points to test	C(n,2) = O(n²)
Side-check per pair (against remaining points)	O(n)
Total Time Complexity	O(n²) × O(n) = O(n³)
Space Complexity	O(n) to store hull boundary points

c) Limitations for Large Datasets

Disaster response systems often work with dense sensor/drone/satellite data covering large affected regions, meaning n can be very large. The cubic O(n³) growth of brute-force convex hull creates serious limitations:

- Even moderate increases in the number of mapped points cause a disproportionately large increase in processing time, delaying urgent boundary-mapping decisions.

- Real-time or near-real-time disaster response cannot tolerate the latency introduced by $O(n^3)$ computation on large point sets.
- Efficient alternatives such as Graham's Scan or the Quickhull algorithm, both averaging O(n log n), are far better suited for large-scale, time-critical disaster mapping.

Question 96: E-Commerce Order Search (Linear Search)

Scenario: An e-commerce system searches for product records stored in an unsorted list using Linear Search.

a) How Linear Search Operates in This System

Linear Search scans the unsorted list of product records sequentially, from the first entry to the last, comparing each record's identifier against the search key until a match is found or the entire list has been examined.

Working steps:

- Start at index 0 of the product list.
- Compare the current product ID with the target ID being searched.
- If it matches, return the current index (record found).
- If not, move to the next index and repeat.
- If the end of the list is reached without a match, report that the product does not exist.

b) Performance Degradation as Data Size Increases

Best Case	O(1) — target found at the very first position
Worst Case	O(n) — target at the last position or not present
Average Case	O(n) — approximately n/2 comparisons on average
Space Complexity	O(1) — no additional memory required

As the e-commerce catalog grows into thousands or millions of products, the number of comparisons required for each search grows proportionally, causing noticeably slower lookup times and a degraded user/customer-service experience during peak traffic.

c) Suggested Improvement and Justification

Switching to Binary Search + Sorting, or using a Hash-based index, would significantly improve performance:

- Sorting the product list by ID once (O(n log n)) and then applying Binary Search reduces each subsequent lookup to O(log n) — a major improvement over O(n) for large catalogs.
- This is worthwhile only if searches are far more frequent than updates, since maintaining sorted order after insertions/deletions adds overhead.
- If updates are equally frequent, a hash table (average O(1) lookup) is often the better choice, trading some memory overhead for near-constant-time search regardless of catalog size.